GIT, GESTION DE PROJET ET AGILITÉ · JOUR 1 SUR 5 · LUNDI 26 OCTOBRE

Lundi · Git sur votre machine

Vous connaissez sûrement ce dossier : `projet_final`, `projet_final_v2`, `projet_final_v2_VRAIMENT_final`. Trois copies, aucune idée de ce qui a changé entre les trois, et pas moyen de revenir à la version qui marchait jeudi dernier.

Git règle ce problème. Il garde l'historique complet de votre projet : chaque version, qui l'a faite, quand et pourquoi. Vous pouvez revenir en arrière, comparer deux versions, et demain, travailler à plusieurs sur les mêmes fichiers sans vous marcher dessus.

Une précision avant de commencer : **Git et GitHub, ce n'est pas la même chose.** Git est un logiciel installé sur votre machine, il marche sans Internet. GitHub est un site web qui héberge des dépôts Git pour les partager. Aujourd'hui, on ne touche qu'à Git. GitHub arrive mardi.

1. Installer et configurer Git

Dans votre VM Ubuntu, vérifiez que Git est là :

```
git --version
```

S'il manque : `sudo apt install git`. Le support est fait avec la version 2.43, celle d'Ubuntu 24.04.

Git signe chaque modification avec votre nom et votre adresse mail. On les règle une fois pour toutes, avec `--global` (valable pour tous vos projets). Utilisez l'adresse que vous mettrez sur GitHub demain.

Configuration, à faire une seule fois

```
$ git config --global user.name "Alice Martin"
$ git config --global user.email "alice@example.com"
$ git config --global init.defaultBranch main
$ git config --global core.editor nano
$ git config --global pull.rebase false
$ git config --global --list
user.name=Alice Martin
user.email=alice@example.com
init.defaultbranch=main
core.editor=nano
pull.rebase=false
```

Les trois dernières lignes évitent des pièges de la semaine. La branche principale s'appellera `main`, comme sur GitHub. Si Git a besoin de vous faire écrire un texte, il ouvrira nano au lieu de vim. Et `pull.rebase false` vous évitera un message d'erreur mercredi.

2. Les trois zones de Git

C'est LA notion de la journée. Un fichier passe par trois endroits :



Pensez à une photo de groupe. `git add`, c'est faire entrer les gens dans le cadre. `git commit`, c'est appuyer sur le déclencheur. Une fois la photo prise, elle est rangée dans l'album pour toujours.

Un **commit**, c'est donc une photo de votre projet à un instant donné, avec un message qui dit ce qui a changé.

3. Votre premier dépôt

On crée un dossier, on dit à Git de le surveiller avec `git init`, et on regarde ce qu'il en pense avec `git status`. Cette dernière commande, tapez-la tout le temps : elle dit toujours où vous en êtes et souvent quoi faire ensuite.

Créer un dépôt et faire le premier commit

```
$ mkdir recettes && cd recettes
$ git init
Initialized empty Git repository in /home/robin/demo/recettes/.git/
$ echo "# Mes recettes" > README.md
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
$ git add README.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

$ git commit -m "Ajoute le README"
[main (root-commit) bed3ced] Ajoute le README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git log --oneline
bed3ced (HEAD -> main) Ajoute le README
```

Lisez les messages de `git status`. Avant le `add`, le fichier est *untracked* (Git ne le suit pas encore). Après, il est dans *Changes to be committed* : il est dans l'index, prêt à partir. Après le commit, `git log` montre la photo, avec un identifiant (`bed3ced`) et le message.

`git init` crée un dossier caché `.git` : c'est là que vit tout l'historique. Ne le supprimez jamais, et ne lancez jamais `git init` dans votre dossier personnel `~` : tous vos fichiers seraient alors surveillés.

4. Modifier, comparer, enregistrer

On ajoute une recette, puis on demande à Git ce qui a changé avec `git diff`. Les lignes qui commencent par `+` ont été ajoutées, celles avec `-` supprimées.

Voir ses modifications avant de les enregistrer

```
$ printf "\n## Crêpes\n250 g de farine, 4 œufs, 50 cl de lait\n" >> README.md
$ echo "Cuire 3 minutes par face." > crepes.txt
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        crepes.txt

no changes added to commit (use "git add" and/or "git commit -a")
$ git diff
diff --git a/README.md b/README.md
index 9d2d26b..759bc70 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,4 @@
 # Mes recettes
+
+## Crêpes
+250 g de farine, 4 œufs, 50 cl de lait
```

`git diff` ne montre que `README.md`. Le fichier `crepes.txt` est nouveau, Git ne le suit pas encore, donc il n'a rien à comparer.

On peut ajouter plusieurs fichiers d'un coup. `git add .` ajoute tout le dossier courant (le point veut dire « ici »). Pratique, mais vérifiez avec `git status` avant de valider.

Enregistrer les deux fichiers dans un commit

```
$ git add README.md crepes.txt
$ git diff --staged
diff --git a/README.md b/README.md
index 9d2d26b..759bc70 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,4 @@
 # Mes recettes
+
+## Crêpes
+250 g de farine, 4 œufs, 50 cl de lait
diff --git a/crepes.txt b/crepes.txt
new file mode 100644
index 0000000..d6e1dd0
--- /dev/null
+++ b/crepes.txt
@@ -0,0 +1 @@
+Cuire 3 minutes par face.
$ git commit -m "Ajoute la recette des crêpes"
[main ee83a2f] Ajoute la recette des crêpes
 2 files changed, 4 insertions(+)
 create mode 100644 crepes.txt
$ git log --oneline
ee83a2f (HEAD -> main) Ajoute la recette des crêpes
bed3ced Ajoute le README
```

`git diff` tout court compare vos fichiers avec l'index. Une fois les fichiers ajoutés, il n'affiche plus rien : il faut `git diff --staged` pour voir ce qui va partir dans le commit.

5. Lire l'historique

Trois façons de lire l'historique

```
$ git log
commit ee83a2fccea0f5a1f6945bf2f4008c80a2e1f780 (HEAD -> main)
Author: Alice Martin <alice@example.com>
Date:   Mon Oct 26 10:00:00 2026 +0100

    Ajoute la recette des crêpes

commit bed3ced20181b97ef760ac8962f7c49d1ddc90dc
Author: Alice Martin <alice@example.com>
Date:   Mon Oct 26 10:00:00 2026 +0100

    Ajoute le README
$ git log --oneline --stat
ee83a2f (HEAD -> main) Ajoute la recette des crêpes
 README.md  | 3 +++
 crepes.txt | 1 +
 2 files changed, 4 insertions(+)
bed3ced Ajoute le README
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git show --stat HEAD
commit ee83a2fccea0f5a1f6945bf2f4008c80a2e1f780 (HEAD -> main)
Author: Alice Martin <alice@example.com>
Date:   Mon Oct 26 10:00:00 2026 +0100

    Ajoute la recette des crêpes

 README.md  | 3 +++
 crepes.txt | 1 +
 2 files changed, 4 insertions(+)
```

`git log` donne le détail de chaque commit : l'auteur, la date et le message. `--oneline` en fait une ligne par commit, `--stat` ajoute les fichiers touchés. `git show` détaille un commit précis. `HEAD` désigne le dernier commit de la branche où vous êtes. Vous pouvez aussi donner un identifiant : `git show bed3ced`.

6. Un bon message de commit

Dans six mois, c'est la seule chose qui vous dira pourquoi vous avez modifié ce fichier. Quelques règles simples :

- un verbe au présent, qui dit ce que fait le commit : « Ajoute la recette des crêpes », « Corrige le calcul de la moyenne » ;
- une cinquantaine de caractères au maximum ;
- **un commit = un changement**. Si votre message contient « et », c'est souvent qu'il fallait deux commits.

À éviter	Mieux
`modif`	`Corrige la division par zéro dans moyenne.sh`
`azerty`	`Ajoute le script du compte à rebours`
`commit du lundi`	`Renomme les exercices de lundi (L01 à L11)`
`ajout palindrome et correction fizzbuzz et readme`	trois commits séparés

Si vous oubliez le `-m`, Git ouvre l'éditeur pour que vous tapiez le message. Avec nano : écrivez le message, puis `Ctrl+O`, `Entrée`, `Ctrl+X`. Si c'est vim qui s'ouvre (config pas faite) : `Échap`, puis tapez `:wq` et `Entrée`.

7. Ignorer des fichiers

Certains fichiers n'ont rien à faire dans l'historique : des notes perso, des fichiers temporaires, des mots de passe. On les liste dans un fichier `.gitignore`, à la racine du dépôt. Une ligne par règle, et `*` veut dire « n'importe quoi ».

Le fichier .gitignore

```
$ echo "pense-bête" > notes-perso.txt
$ echo "erreur 42" > debug.log
$ git status --short
?? debug.log
?? notes-perso.txt
$ printf "notes-perso.txt\n*.log\n" > .gitignore
$ git status --short
?? .gitignore
$ git add .gitignore
$ git commit -m "Ignore les notes perso et les logs"
[main defaf76] Ignore les notes perso et les logs
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```

`git status --short` est la version courte : `??` veut dire « pas suivi ». Après le `.gitignore`, les deux fichiers disparaissent de la liste. Seul `.gitignore` lui-même reste à ajouter.

Attention : un fichier **déjà commité** n'est pas ignoré, même s'il est dans `.gitignore`. Il faut d'abord le retirer du suivi avec `git rm --cached fichier`.

8. Revenir en arrière

Trois situations, trois commandes.

Situation	Commande	Ce qui se passe
J'ai modifié un fichier, je veux annuler (pas encore de `add`)	`git restore fichier`	le fichier revient à son état du dernier commit. **Vos modifications sont perdues.**
J'ai fait `add` par erreur	`git restore --staged fichier`	le fichier sort de l'index, vos modifications restent
J'ai commité une bêtise	`git revert <id>`	Git crée un **nouveau** commit qui annule l'ancien

Annuler une modification avec restore

```
$ echo "N'IMPORTE QUOI" > crepes.txt
$ git status --short
 M crepes.txt
$ git restore crepes.txt
$ cat crepes.txt
Cuire 3 minutes par face.
```

Annuler un commit avec revert

```
$ echo "Recette secrète de mamie" >> README.md
$ git commit -am "Ajoute la recette secrète"
[main eb102de] Ajoute la recette secrète
 1 file changed, 1 insertion(+)
$ git log --oneline
eb102de (HEAD -> main) Ajoute la recette secrète
defaf76 Ignore les notes perso et les logs
ee83a2f Ajoute la recette des crêpes
bed3ced Ajoute le README
$ git revert --no-edit HEAD
[main 883729d] Revert "Ajoute la recette secrète"
 Date: Mon Oct 26 10:00:00 2026 +0100
 1 file changed, 1 deletion(-)
$ git log --oneline
883729d (HEAD -> main) Revert "Ajoute la recette secrète"
eb102de Ajoute la recette secrète
defaf76 Ignore les notes perso et les logs
ee83a2f Ajoute la recette des crêpes
bed3ced Ajoute le README
$ tail -n 1 README.md
250 g de farine, 4 œufs, 50 cl de lait
```

`git commit -am` fait le `add` et le `commit` d'un coup, mais seulement pour les fichiers que Git suit déjà. Un nouveau fichier doit toujours passer par `git add`.

Avec `revert`, l'erreur reste visible dans l'historique, suivie de son annulation. On n'efface pas le passé, on le corrige. Ça compte quand on travaille à plusieurs : réécrire un historique que les autres ont déjà récupéré crée des catastrophes. Vous croiserez `git reset --hard` sur Internet. Il efface des commits et des modifications sans retour possible : on ne l'utilise pas cette semaine.

Exercices du lundi

Pour ces exercices, vous allez versionner vos scripts de la semaine dernière. Récupérez-les dans le dossier du cours ou recopiez-en quelques-uns.

L1. Configurer Git NIVEAU 1

Faites la configuration de la section 1 avec **votre** nom et **votre** adresse mail. Vérifiez :

```
git config --global --list
```

Vous devez voir `user.name`, `user.email`, `init.defaultbranch=main`, `core.editor=nano` et `pull.rebase=false`.

L2. Versionner vos scripts bash NIVEAU 1

1. Créez un dossier `algo-bash` dans votre home et placez-vous dedans.
2. Lancez `git init`.
3. Copiez-y au moins trois de vos scripts de la semaine dernière.
4. Regardez `git status`, ajoutez les scripts, et faites un commit « Ajoute mes exercices de bash ».
5. Vérifiez avec `git log --oneline`.

Votre identifiant de commit sera différent de ceux du support : il dépend de votre nom, de la date et du contenu.

L3. Un commit par changement NIVEAU 2

Faites deux modifications **sans rapport** : ajoutez un commentaire en haut d'un premier script, et corrigez ou améliorez l'affichage d'un deuxième. Puis faites **deux commits séparés**, un par modification. L'astuce : `git add` un seul fichier à la fois.

Vérifiez avec `git log --oneline --stat` : chaque commit ne doit toucher qu'un seul fichier.

L4. Le détective NIVEAU 2

Sur votre dépôt `algo-bash`, répondez aux questions en utilisant seulement Git :

1. Combien de commits contient votre dépôt ? (`git log --oneline`, puis comptez, ou ajoutez `| wc -l`)
2. Quels fichiers ont été modifiés par votre deuxième commit ? (`git show --stat <id>`)
3. Qu'est-ce qui a changé entre votre premier et votre dernier commit ? (`git diff <id1> <id2>`)

L5. Le fichier .gitignore NIVEAU 2

1. Créez un fichier `brouillon.txt`, un fichier `test.log` et un dossier `tmp/` avec un fichier dedans.
2. Écrivez un `.gitignore` pour que Git ignore les trois. Pour un dossier entier, la règle est `tmp/`.
3. `git status --short` ne doit plus montrer que `?? .gitignore`.
4. Commitez le `.gitignore`.

L6. Réparer ses erreurs NIVEAU 2

Trois catastrophes à provoquer, puis à réparer.

1. Remplacez tout le contenu d'un script par « oups ». Faites-le revenir avec `git restore`.
2. Modifiez un script, faites `git add`, puis changez d'avis : sortez-le de l'index avec `git restore --staged`. Vérifiez que votre modification est toujours dans le fichier.
3. Ajoutez une ligne `echo "bug"` à la fin d'un script et **commitez**. Annulez ce commit avec `git revert`, et regardez `git log --oneline` : combien de commits voyez-vous de plus qu'avant ?

L7. Chercher dans l'historique NIVEAU 3

Deux commandes pour enquêter :

- `git log -S "texte" --oneline` liste les commits qui ont ajouté ou retiré ce texte ;
- `git blame fichier` affiche, pour chaque ligne du fichier, le commit et l'auteur qui l'ont écrite en dernier.

Trouvez avec `git log -S` le commit qui a ajouté votre `echo "bug"` de L6, puis celui qui l'a retiré. Lancez ensuite `git blame` sur un de vos scripts.

Mémo Git

Besoin	Commande
Créer un dépôt dans le dossier courant	`git init`
Où j'en suis ?	`git status` (version courte : `git status -s`)
Ajouter des fichiers au prochain commit	`git add fichier`, ou `git add .` pour tout
Enregistrer un commit	`git commit -m "Message au présent"`
Ajouter et commiter les fichiers déjà suivis	`git commit -am "Message"`
Voir ce qui a changé (pas encore ajouté)	`git diff`
Voir ce qui va partir dans le commit	`git diff --staged`
Historique	`git log --oneline`, `git log --oneline --graph --all`
Détail d'un commit	`git show <id>`
Annuler une modification non ajoutée	`git restore fichier` (perd la modification)
Sortir un fichier de l'index	`git restore --staged fichier`
Annuler un commit sans réécrire l'historique	`git revert <id>`
Lister les branches	`git branch`
Créer une branche et y aller	`git switch -c nom`
Changer de branche	`git switch nom`
Fusionner une branche dans la branche courante	`git merge nom`
Supprimer une branche fusionnée	`git branch -d nom`
Abandonner une fusion en conflit	`git merge --abort`
Relier le dépôt à GitHub	`git remote add origin git@github.com:pseudo/depot.git`
Premier envoi d'une branche	`git push -u origin nom`
Envoyer / récupérer	`git push` / `git pull`
Télécharger un dépôt	`git clone <adresse>`
Récupérer sans fusionner	`git fetch`
Qui a écrit cette ligne ?	`git blame fichier`
Quel commit a ajouté ce texte ?	`git log -S "texte" --oneline`

Dans un conflit, le fichier contient `<<<<<<< HEAD` (votre version), `=======`, puis `>>>>>>> branche` (l'autre version). Gardez ce que vous voulez, supprimez les trois marqueurs, puis `git add` et `git commit`.